

Quantization Analysis of Transformer Architectures with a CNN Reference

Sahil Bhatt

Research in Computer and Systems Engineering
Technische Universität Ilmenau
sahil.bhatt@tu.ilmenau.de

Abstract—Transformer models achieve strong results in language and vision tasks, but their parameter counts and memory demands complicate efficient deployment. This group study analyzes weight quantization in two transformer architectures—DistilBERT and ViT-B/16—with ResNet-18 as a CNN reference. It presents research-informed custom implementations of symmetric per-channel post-training quantization (PTQ) and non-uniform codebook quantization, without fake quantization or retraining. Experiments use the complete CIFAR-10 test split and labelled SST-2 development split. Two-to-eight-bit sweeps measure accuracy, compression ratio, and parameter-memory reduction, while four- and eight-bit layer or module studies measure isolated sensitivity and cross-bit rank stability. At eight bits, the maximum accuracy degradation across all three models and both custom methods is 0.59 percentage points, with approximately 75% parameter-storage reduction. At four bits, DistilBERT and ViT remain within 0.6 percentage points of their FP32 baselines while achieving about $7.9\times$ compression. ResNet-18 instead falls from 90.47% to 11.56% with PTQ and 8.43% with codebook quantization; quantizing its first convolution alone causes losses of 60.65 and 78.62 percentage points. These results show that the evaluated transformers tolerate four-bit weight quantization substantially better than the CNN reference, while very low bit widths remain method and architecture dependent.

Index Terms—neural-network quantization, post-training quantization, vector quantization, codebook quantization, layer sensitivity, ResNet, DistilBERT, Vision Transformer

I. Introduction

Deep neural networks achieve strong performance in computer vision and natural-language processing, but their parameter storage, memory traffic, and arithmetic requirements complicate deployment on mobile, embedded, and edge systems. Quantization addresses these costs by representing weights, activations, or both with fewer bits than FP32 [1]–[3]. Replacing 32-bit parameters by packed 8-bit or 4-bit values suggests ideal storage reductions of four or eight times. Actual savings are lower because scales, codebooks, alignment, and preserved parameters must also be stored. More importantly, low precision introduces numerical error that can change model decisions.

The resulting accuracy cost is architecture dependent. A convolutional network constructs hierarchical visual features through local kernels. A language transformer uses token embeddings, attention projections, and feed-forward matrices. A vision transformer converts image patches into tokens before applying transformer encoders.

These components differ in parameter distribution, function, and position in the inference path, and they need not tolerate the same quantization policy. A full-model accuracy number also does not reveal whether failure is distributed across the network or dominated by a small number of sensitive components.

This project studies two post-training, weight-only techniques. The first is symmetric uniform PTQ with per-output-channel scaling. The second is non-uniform codebook quantization, in which each weight is represented by a low-bit index into a learned set of centroids. Both are research-informed custom implementations: the code is original to this project, while the mathematical principles are based on established literature. No new quantization algorithm is claimed. No fake-quantization layers or quantization-aware retraining are used.

This group study focuses primarily on quantization in transformer-based artificial neural networks. DistilBERT represents a language transformer and ViT-B/16 represents a vision transformer; ResNet-18 provides a CNN reference for identifying architecture-dependent behavior. The experimental design combines full-model bit-width sweeps with isolated layer, tensor, or module experiments in which only one analysis unit is quantized while all remaining weights stay FP32. Accuracy and parameter representation are measured directly. Computational efficiency is discussed only through parameter storage because the custom evaluation path reconstructs weights for floating-point Keras execution and does not benchmark latency, throughput, energy, or runtime memory.

A. Research Questions

The study addresses the following questions:

- 1) How do bit width and quantization method affect accuracy, compression ratio, and parameter-memory reduction?
- 2) How do DistilBERT and ViT-B/16 differ from the ResNet-18 CNN reference at four and eight bits?
- 3) Which layers, tensors, or modules cause the largest accuracy degradation when quantized in isolation?
- 4) Does an eight-bit sensitivity ranking reliably predict four-bit sensitivity?

B. Scope and Deliverables

The project scope and principal outputs are:

- reusable mathematical n -bit PTQ and codebook weight-quantization implementations with explicit storage accounting;
- a consistent evaluation across a CNN, a language transformer, and a vision transformer;
- complete full-evaluation-split two-to-eight-bit PTQ and codebook sweeps for all three architectures;
- exhaustive W8 and exhaustive or architecture-stratified W4 sensitivity studies across convolutional and transformer models;
- reproducible notebooks with fixed selection policies, cached predictions, CSV tables, figures, and run metadata; and
- an architecture-level comparison that identifies the advantages and limitations of low-bit weight quantization in the evaluated transformer and CNN models.

II. Transformer Architectures, Models, and Datasets

The evaluation covers three model families and two data modalities. Each quantized model is derived from the same saved task-specific FP32 checkpoint used for its baseline. Accuracy degradation is therefore measured relative to the FP32 version of the same architecture and task, not by comparing absolute accuracies across different datasets.

A. Transformer Architecture Context

A transformer encoder represents its input as a sequence of token embeddings and repeatedly applies multi-head self-attention [4] and a position-wise feed-forward network. For one attention head, input projections form query, key, and value matrices Q , K , and V , and scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (1)$$

where d_k is the key dimension [4]. Multiple heads learn different projection subspaces, after which their outputs are concatenated and projected. Residual connections, normalization, positional information, and feed-forward layers complete each encoder block. The large embedding, attention-projection, and feed-forward matrices are therefore natural targets for weight quantization.

DistilBERT converts subword tokens into learned token and position embeddings and processes them with six transformer layers for sentence classification. ViT-B/16 instead converts image patches into token embeddings, prepends a learned class token, adds positional embeddings, and processes the resulting sequence with 12 transformer blocks. Although their input modalities differ, both models rely primarily on the same attention and feed-forward matrix operations. This shared structure enables a consistent quantization study, while ResNet-18 provides a convolution-based reference with local kernels and hierarchical spatial features.

B. ResNet-18 and CIFAR-10

ResNet-18 is built from residual blocks whose shortcut connections allow a block to learn a residual transformation relative to its input [15]. It is selected as the CNN reference because its progression from an input convolution through residual stages to a classification head provides clearly defined kernels for layer-wise analysis. The model was initialized from the KerasHub `resnet_18_imagenet` preset and adapted to CIFAR-10 with a ten-class output layer. The checkpoint contains 11,191,242 parameters.

CIFAR-10 contains 60,000 colour images of size 32×32 in ten balanced classes, divided into 50,000 training and 10,000 test images [16]. Evaluation uses the complete official test split. Images are resized to 224×224 and normalized using the preset’s preprocessing; no test-time augmentation is applied. The FP32 checkpoint correctly classifies 9,047 test images, giving 90.47% accuracy.

C. DistilBERT and SST-2

DistilBERT is a compact language transformer obtained by distilling BERT into six transformer layers [17]. Its trainable components include token and positional embeddings, self-attention projections, feed-forward matrices, normalization parameters, and a classification head. The `distil_bert_base_en_uncased` preset supplies the pretrained backbone, which is fine-tuned with a binary sentiment head. The saved classifier contains 66,955,010 parameters.

SST-2 assigns a positive or negative label to each sentence and is distributed through the GLUE benchmark [18], [19]. Evaluation uses all 872 labelled development examples because official test labels are not public. The matching tokenizer produces token identifiers and padding masks of fixed length 128. The FP32 model correctly predicts 771 examples, giving 88.42% accuracy. Since one example corresponds to approximately 0.115 pp, small changes must be interpreted with the sample size in mind.

D. ViT-B/16 and CIFAR-10

The Vision Transformer represents an image as a sequence of patch embeddings and applies transformer encoder blocks instead of convolutional feature-extraction stages [20]. This study uses the `vit_base_patch16_224_imagenet` preset, operating on 224×224 images with 16×16 patches. An ImageNet-pretrained backbone is combined with a ten-class CIFAR-10 head. The backbone remained frozen while the task head was trained. The resulting classifier contains 85,806,346 parameters.

ViT is evaluated on the same 10,000 CIFAR-10 test images and labels as ResNet-18, using its own preset-specific resizing and normalization. The FP32 checkpoint correctly classifies 9,573 images, giving 95.73% accuracy. Sharing the dataset supports a CNN–vision-transformer comparison, although the models have different pretraining and adaptation procedures.

TABLE I
Models, datasets, and FP32 reference measurements

Model	Role	Dataset split	Input	Classes	Examples	Parameters	Size (MiB)	Accuracy
ResNet-18	CNN reference	CIFAR-10 test	$224 \times 224 \times 3$	10	10,000	11,191,242	42.69	90.47%
DistilBERT	Language transformer	SST-2 development	128 tokens	2	872	66,955,010	255.41	88.42%
ViT-B/16	Vision transformer	CIFAR-10 test	$224 \times 224 \times 3$	10	10,000	85,806,346	327.33	95.73%

Table I reports parameter-array storage: the sum of the in-memory bytes of all FP32 parameter arrays. It is not the size of a serialized .keras archive, which may contain additional metadata or compression. These FP32 values are the denominators for all later compression and memory-reduction calculations.

III. Quantization Methods and Research Foundations

The methods are described as research-informed custom implementations. Their software implementation and experimental integration are project work, but their mathematical foundations come from prior studies. Jacob et al. establish affine integer quantization and integer accumulation [1]; Krishnamoorthi discusses post-training calibration and per-channel weight quantization [2]; and Nagel et al. provide a general PTQ framework and analysis of low-bit failure modes [3]. The codebook path draws from classical vector quantization [5], product quantization [8], neural-network vector quantization [9], weight sharing [10], and sensitivity-weighted distortion [11]. Optional library functions are additionally informed by Diffusion Product Quantization (DPQ) [12] and VQ4ALL [13]. The principal experiments, however, use tensor-local scalar codebooks rather than the complete training procedures proposed by DPQ or VQ4ALL.

A. Symmetric Per-Channel PTQ

An affine quantizer maps a real value x to an integer q using scale s and zero point z :

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right), \quad (2)$$

with reconstruction

$$\hat{x} = s(q - z). \quad (3)$$

The custom weight path is signed and symmetric, so $z = 0$. For bit width b ,

$$Q_b = 2^{b-1} - 1, \quad q_{\min} = -Q_b, \quad q_{\max} = Q_b. \quad (4)$$

For output channel c , the scale and integer weights are

$$s_c = \frac{\max_i |w_{c,i}|}{Q_b}, \quad (5)$$

$$q_{c,i} = \text{clip}\left(\text{round}\left(\frac{w_{c,i}}{s_c}\right), -Q_b, Q_b\right).$$

The last tensor axis is treated as the output-channel axis. If a channel is all zero, a safe nonzero scale is used and its reconstruction remains zero. The quantizer stores

the packed integers and FP32 channel scales needed for reconstruction.

The codebase also supports representative-data activation-range calibration and explicit integer dense/convolution arithmetic. Those functions are not used in the custom accuracy sweeps reported here. The evaluated custom models reconstruct the low-bit weights into floating arrays and execute with FP32 activations in Keras.

B. Codebook Quantization

Codebook quantization represents weights through indices into learned centroids. Given scalar or vector groups v_i and codebook $C = \{c_1, \dots, c_K\}$, the assignment and reconstruction are

$$a_i = \arg \min_{k \in \{1, \dots, K\}} \|v_i - c_k\|_2^2, \quad \hat{v}_i = c_{a_i}. \quad (6)$$

The centroids are fitted with deterministic k-means++ initialization [6] followed by Lloyd updates [7]. Large tensors may be sampled for codebook fitting, but exact nearest-centroid assignment is then performed for every selected value. A b -bit experiment uses $K = 2^b$ centroids, requiring b assignment bits.

The main report uses scalar groups ($d = 1$). This makes the assignment width directly comparable with a b -bit PTQ weight. The implementation also supports vector and product modes, importance-weighted k-means, shared KDE-generated codebooks, and product-codebook pooling. These capabilities reflect ideas in Jégou et al., Choi et al., DPQ, and VQ4ALL [8], [11]–[13], but they are not presented as evaluated contributions unless explicitly identified in a result.

C. Tensor Selection and Evaluation Execution

Tensor selection is identical for PTQ and codebook quantization within each experiment. A tensor must first be floating point and have rank at least two. This rank test is the complete ResNet selector, but it is only a necessary condition for the transformer experiments. DistilBERT additionally requires the complete variable path to end in /kernel or /embeddings. ViT accepts paths ending in /kernel or /embeddings, together with the learned class_token. These explicit name filters prevent rank-two attention biases from being selected merely because of their shape. Table II records the resulting coverage.

The 40 DistilBERT tensors comprise token and position embeddings, the query, key, value, and output kernels of each attention layer, both feed-forward kernels in each

TABLE II
Exact full-model tensor-selection policy used by both custom methods

Model	Full-model selector	Selected
ResNet-18	Floating tensor with rank ≥ 2	21
DistilBERT	Floating tensor with rank ≥ 2 and path ending in /kernel or /embeddings	40
ViT-B/16	Floating tensor with rank ≥ 2 and path ending in /kernel or /embeddings, or named class_token	76

transformer layer, the pooled projection, and the classifier kernel. The 76 ViT tensors comprise the patch-embedding kernel, position embedding, class token, six kernels in each of the 12 transformer blocks (four attention projections and two MLP kernels), and the classifier kernel. Rank-one biases and normalization scale/offset vectors remain FP32 in all three models. Transformer attention biases also remain FP32 even where their implementation gives them rank two. Leaving these tensors unquantized is an experimental engineering policy, not a theoretical requirement.

Every evaluated configuration starts from the same ordered FP32 weight list. For PTQ, each selected tensor is converted to integer values and scale metadata and then reconstructed using the inverse quantization equation. For codebook quantization, the selected tensor is reconstructed by replacing each stored assignment with its FP32 centroid. The reconstructed array is cast to the original tensor dtype and inserted at the same weight index in a Keras evaluation model; every unselected tensor is copied unchanged from the FP32 checkpoint. A full-model result replaces all selected tensors simultaneously. Quantization is never applied cumulatively from one bit width or configuration to the next.

The custom evaluation path executes the reconstructed model using ordinary Keras floating-point operators and FP32 activations. It does not execute packed 2–8-bit weights, integer accumulators, or indexed codebook lookups during inference. Storage figures instead estimate a separate deployable representation: bit-packed integer values or assignments, required FP32 scales or centroids, and all preserved FP32 tensors. Thus, the reported custom compression is parameter-representation compression rather than measured Keras runtime-memory compression. No custom latency, energy, throughput, or runtime-RAM improvement is claimed. This distinction is particularly important for non-uniform codebooks, whose storage savings do not automatically translate into efficient fixed-point matrix multiplication [14]. The built-in TensorFlow Lite baselines are evaluated separately: the ResNet baseline is executable W8A8, whereas the transformer dynamic-range baselines retain FP32 activations.

IV. Experimental Methodology and Results

The experimental evaluation first measures complete-model behaviour and then investigates which internal

TABLE III
FP32 evaluation on the complete labelled evaluation splits

Model	Examples	Correct	Accuracy
ResNet-18	10,000	9,047	90.47%
DistilBERT	872	771	88.42%
ViT-B/16	10,000	9,573	95.73%

components cause the observed degradation. Each configuration is generated independently from the saved task-specific FP32 checkpoint and is evaluated on the complete labelled evaluation split. The reported size is parameter-representation size rather than serialized file size or peak runtime memory. Compression ratio R and parameter-memory reduction S are calculated as

$$R = \frac{M_{\text{FP32}}}{M_q}, \quad S = 100 \left(1 - \frac{M_q}{M_{\text{FP32}}} \right). \quad (7)$$

A. FP32 Full-Model Evaluation

The first experiment establishes the FP32 prediction baseline for each architecture. ResNet-18 and ViT-B/16 are evaluated on all 10,000 images in the official CIFAR-10 test split. DistilBERT is evaluated on all 872 labelled SST-2 development examples. Each checkpoint is evaluated without modifying its parameters and without test-time augmentation. Table III reports the resulting reference accuracies. All later accuracy differences, compression ratios, and memory reductions use the corresponding FP32 model as their baseline.

B. Eight-Bit Full-Model Quantization

The second experiment quantizes all tensors selected by the model-specific policy in Table II. Custom PTQ and codebook results use W8 weights reconstructed for Keras evaluation with FP32 activations. The built-in TensorFlow Lite comparison is executable W8A8 for ResNet-18. The DistilBERT and ViT dynamic-range models use supported W8 constants with FP32 activations. The built-in rows are therefore software baselines rather than numerically identical implementations of the custom weight-only experiment.

Table IV shows that W8 quantization is stable for all three architectures. Across the custom methods, the largest accuracy loss is 0.59 percentage points for ResNet PTQ. All custom W8 representations are approximately

4 $\times$ smaller than FP32 and reduce parameter storage by about 75%.

A negative drop means that the quantized model classified a small number of additional evaluation examples correctly. It is reported as an observed result and is not interpreted as evidence that quantization improves generalization.

C. Full Two-to-Eight-Bit Sweeps

For every sweep point, the selected FP32 weights are quantized directly to the target bit width. The 3-bit model is therefore generated from FP32 rather than from the 2-bit model, and the same rule is applied through eight bits. Every point is evaluated on the same complete split used for its FP32 baseline. Notebook 25 consolidates 42 measured configurations: seven bit widths, two methods, and three architectures. No sweep point is interpolated or estimated.

Figure 1 compares the three architectures separately for PTQ and codebook quantization. The dashed lines mark each model’s FP32 accuracy. Absolute accuracy across DistilBERT and the image models should not be interpreted as a task-level ranking because the datasets differ; the relevant comparison is the shape and breakpoint of each model’s bit-width response.

ResNet-18 exhibits a severe low-bit transition under both methods (Table V). PTQ remains near chance from two through four bits, then rises to 49.39% at five bits, 71.87% at six bits, and 88.05% at seven bits. Codebook quantization recovers later: it reaches only 48.71% at six bits but rises to 88.97% at seven bits and 90.25% at eight bits. Thus, W8 codebook is slightly closer to FP32 than W8 PTQ, but neither method makes full-model W4 ResNet usable.

DistilBERT behaves differently (Table VI). At two bits, codebook quantization retains 78.33% accuracy compared with 51.03% for PTQ. At three bits, the gap narrows to 86.24% versus 82.91%. From four through eight bits, both methods remain within 0.58 percentage points of FP32.

The W4 PTQ result is 88.99%, compared with the 88.42% FP32 baseline: 776 rather than 771 correctly classified examples. This five-example difference is reported as an observed fluctuation on the small SST-2 development split and is not interpreted as evidence that quantization improves generalization.

ViT exhibits a stronger low-bit method interaction (Table VII). At two bits, codebook accuracy is 20.84% and PTQ accuracy is 13.00%. At three bits, codebook quantization reaches 89.77%, whereas PTQ reaches only 32.22%. Both recover abruptly at four bits, reaching 95.20% and 95.23%, respectively. From four through eight bits, both methods remain close to the 95.73% FP32 baseline.

Across the completed sweeps, compression is governed mainly by bit width because the selected tensors contain

most model parameters. Two-bit representations provide approximately 15.5–15.8 $\times$ compression and about 93.6% parameter-memory reduction. Four-bit representations provide approximately 7.9 $\times$ compression and about 87.3% reduction. Eight-bit representations provide approximately 4 $\times$ compression and about 74.8–74.9% reduction. Accuracy, unlike storage, remains strongly dependent on both architecture and quantization method.

Figure 2 consolidates accuracy, compression ratio, and parameter-memory reduction for all 42 configurations. The near-overlap of compression curves confirms that storage is determined primarily by bit width, whereas the separated accuracy curves expose architecture- and method-specific numerical sensitivity.

TABLE IV
Eight-bit full-model accuracy and parameter representation. The ResNet built-in row is W8A8; the transformer built-in rows retain FP32 activations.

Model	Representation	Accuracy	Drop (pp)	Size (MiB)	Compression	Reduction
ResNet-18	FP32	90.47%	0.00	42.69	1.00×	0.00%
	Built-in full INT8	88.53%	1.94	10.67	4.00×	75.00%
	Custom per-channel W8	89.88%	0.59	10.73	3.98×	74.87%
	Custom codebook W8	90.25%	0.22	10.75	3.97×	74.82%
DistilBERT	FP32	88.42%	0.00	255.41	1.00×	0.00%
	Built-in dynamic W8	88.07%	0.34	64.03	3.99×	74.93%
	Custom per-channel W8	88.42%	0.00	64.15	3.98×	74.88%
	Custom codebook W8	88.07%	0.34	64.07	3.99×	74.92%
ViT-B/16	FP32	95.73%	0.00	327.33	1.00×	0.00%
	Built-in dynamic W8	95.78%	-0.05	82.62	3.96×	74.76%
	Custom per-channel W8	95.79%	-0.06	82.41	3.97×	74.82%
	Custom codebook W8	95.73%	0.00	82.25	3.98×	74.87%

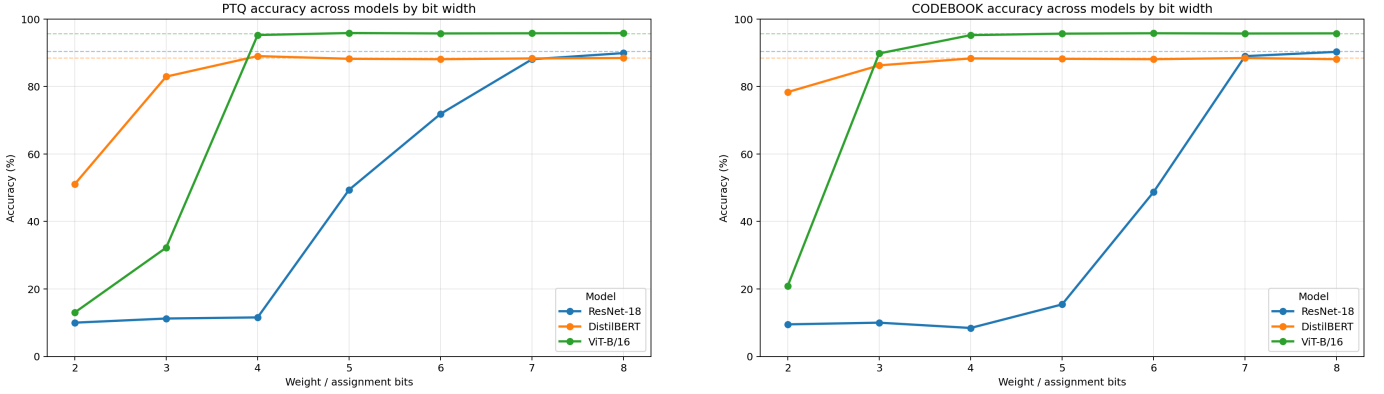


Fig. 1. Bit-wise accuracy across all three architectures for PTQ (left) and codebook quantization (right). Dashed lines show the corresponding FP32 baselines.

TABLE V
ResNet-18 PTQ and codebook sweep. A is accuracy, R is compression ratio, and S is parameter-memory reduction.

Bits	PTQ A	PTQ R	PTQ S	Codebook A	Codebook R	Codebook S
2	10.00%	15.60×	93.59%	9.50%	15.60×	93.59%
3	11.23%	10.49×	90.47%	10.00%	10.49×	90.47%
4	11.56%	7.91×	87.35%	8.43%	7.90×	87.35%
5	49.39%	6.34×	84.23%	15.41%	6.34×	84.22%
6	71.87%	5.29×	81.11%	48.71%	5.29×	81.10%
7	88.05%	4.54×	77.99%	88.97%	4.54×	77.97%
8	89.88%	3.98×	74.87%	90.25%	3.97×	74.82%

TABLE VI
DistilBERT PTQ and codebook sweep. A is accuracy, R is compression ratio, and S is parameter-memory reduction.

Bits	PTQ A	PTQ R	PTQ S	Codebook A	Codebook R	Codebook S
2	51.03%	15.67×	93.62%	78.33%	15.78×	93.66%
3	82.91%	10.52×	90.49%	86.24%	10.57×	90.54%
4	88.99%	7.92×	87.37%	88.30%	7.95×	87.42%
5	88.19%	6.35×	84.25%	88.19%	6.37×	84.29%
6	88.07%	5.30×	81.13%	88.07%	5.31×	81.17%
7	88.30%	4.55×	78.01%	88.42%	4.55×	78.04%
8	88.42%	3.98×	74.88%	88.07%	3.99×	74.92%

TABLE VII
ViT-B/16 PTQ and codebook sweep. A is accuracy, R is compression ratio, and S is parameter-memory reduction.

Bits	PTQ A	PTQ R	PTQ S	Codebook A	Codebook R	Codebook S
2	13.00%	15.50×	93.55%	20.84%	15.66×	93.62%
3	32.22%	10.45×	90.43%	89.77%	10.52×	90.50%
4	95.23%	7.88×	87.31%	95.20%	7.92×	87.37%
5	95.83%	6.32×	84.19%	95.64%	6.35×	84.25%
6	95.70%	5.28×	81.06%	95.76%	5.30×	81.13%
7	95.75%	4.53×	77.94%	95.68%	4.55×	78.00%
8	95.79%	3.97×	74.82%	95.73%	3.98×	74.87%

Full-model PTQ and codebook quantization: 2--8-bit trade-offs

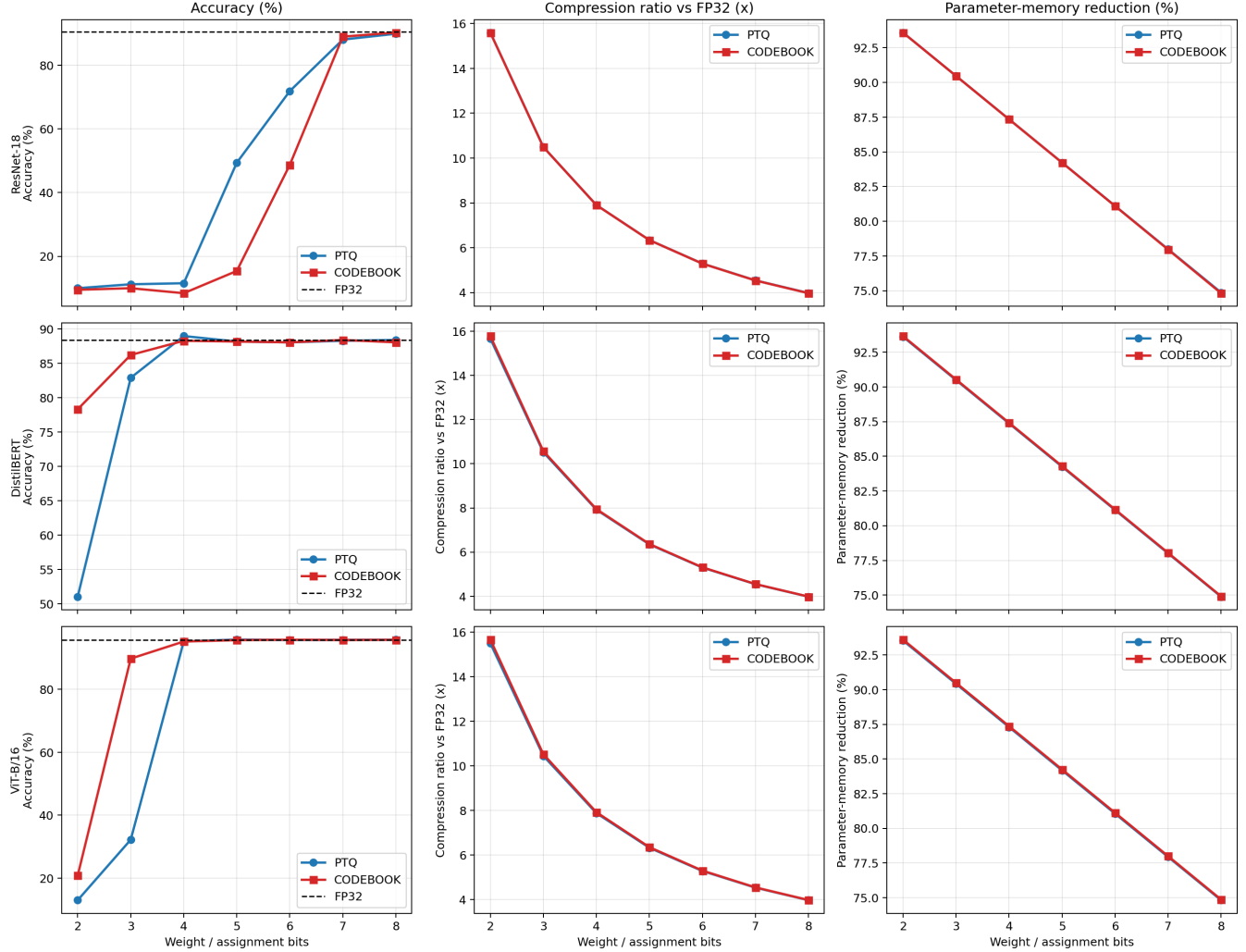


Fig. 2. Consolidated two-to-eight-bit results from Notebook 25. Rows correspond to ResNet-18, DistilBERT, and ViT-B/16; columns show accuracy, compression ratio, and parameter-memory reduction.

D. Layer and Module Sensitivity Analysis

1) Motivation and experimental protocol: The full-model sweeps identify when quantization fails, but they do not identify which internal components cause the failure. The sensitivity experiments therefore address two questions: which units are most sensitive, and whether a W8 sensitivity ranking transfers reliably to W4.

For an isolated sensitivity run, the experiment starts from the original FP32 checkpoint, quantizes exactly one analysis unit, leaves every other parameter in FP32, and evaluates the resulting model on the complete labelled

evaluation split. ResNet uses individual eligible kernels, DistilBERT uses individual eligible embedding or kernel tensors, and ViT groups its 76 eligible tensors into 14 logical modules. The accuracy degradation for unit u is $\Delta A_u = A_{FP32} - A_u$, where A_u is the accuracy when only unit u is quantized. A negative value means that the configuration happens to classify more evaluation examples correctly; it is not treated as evidence of improved generalization.

All configurations use the complete evaluation split. ResNet is exhaustive at W8 and W4 (all 21 kernels). DistilBERT covers all 40 tensors at W8 and a predefined architecture-stratified subset of 15 at W4; ViT covers all 14 modules at W8 and a predefined stratified subset of seven at W4. The subsets were fixed before inspecting W4 outcomes.

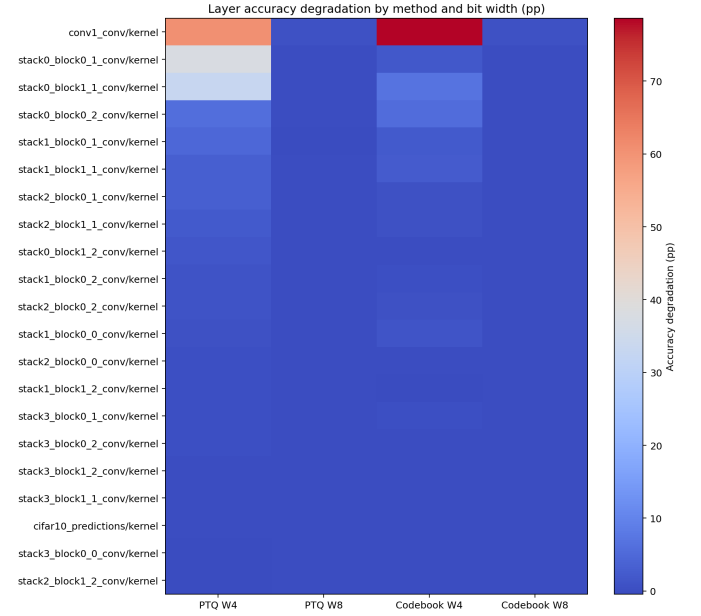


Fig. 3. Matched ResNet-18 isolated kernel sensitivity at W4 and W8 for PTQ and codebook quantization. Only one kernel is quantized in each evaluation.

2) Isolated W8 and W4 sensitivity: Table VIII reports the largest isolated accuracy loss for every architecture, method, and precision. At W8, all three architectures are stable: the largest isolated loss is only 0.53 pp. At W4, the transformer results remain small over their stratified samples, whereas ResNet becomes highly non-uniform. Quantizing only its first convolution reduces accuracy by 60.65 pp under PTQ and 78.62 pp under codebook quantization. Thirteen of 21 W4 codebook kernels and 14 of 21 W4 PTQ kernels remain significant after false-discovery-rate correction. No sampled DistilBERT or ViT unit is FDR-significant.

TABLE VIII
Maximum isolated accuracy degradation. Transformer W4 values refer to predefined stratified samples; “Sig.” gives the FDR-significant count.

Model	Method	Bits	Drop (pp)	Most sensitive unit	Sig.
ResNet-18	PTQ	8	0.53	First convolution	1/21
ResNet-18	Codebook	8	0.52	First convolution	1/21
ResNet-18	PTQ	4	60.65	First convolution	14/21
ResNet-18	Codebook	4	78.62	First convolution	13/21
DistilBERT	PTQ	8	0.115	L1 attention value	0/40
DistilBERT	Codebook	8	0.229	L2 intermediate FFN	0/40
DistilBERT	PTQ	4	0.115	L2 intermediate FFN	0/15
DistilBERT	Codebook	4	0.229	L2 intermediate FFN	0/15
ViT-B/16	PTQ	8	0.02	Transformer block 4	0/14
ViT-B/16	Codebook	8	0.06	Transformer block 6	0/14
ViT-B/16	PTQ	4	0.04	Transformer block 6	0/7
ViT-B/16	Codebook	4	0.20	Transformer block 1	0/7

Figure 3 visualizes the ResNet result. W8 perturbations are small across all kernels, while W4 exposes a small set of severe early-layer failures. This provides a component-level explanation for the full-model W4 collapse observed in Table V.

3) Cross-bit rank stability: The next experiment tests whether the W8 sensitivity order predicts behavior at W4. Spearman rank correlations are calculated over matched units. Table IX shows that rank transfer is generally unreliable. ResNet and ViT correlations are negative and non-significant, and DistilBERT PTQ is also non-significant. DistilBERT codebook quantization is the only tested case with a statistically significant positive relationship ($\rho = 0.693$, $p = 0.004$). Consequently, W8 sensitivity should not be assumed to identify W4-sensitive components without validation.

TABLE IX
W4/W8 sensitivity-rank stability

Model	Method	Spearman ρ	p -value
ResNet-18	PTQ	-0.094	0.687
ResNet-18	Codebook	-0.292	0.199
DistilBERT	PTQ	0.143	0.612
DistilBERT	Codebook	0.693	0.004
ViT-B/16	PTQ	-0.321	0.482
ViT-B/16	Codebook	-0.357	0.432

V. Limitations

Statistical significance for isolated sensitivity experiments was assessed using paired McNemar tests [21], followed by Benjamini–Hochberg false-discovery-rate correction across the tested units [22]. Spearman rank correlation was used to compare W4 and W8 sensitivity rankings.

Three limitations qualify the conclusions. First, the DistilBERT and ViT W4 isolated studies cover predefined architecture-stratified subsets rather than all eligible units. Second, DistilBERT is evaluated on the relatively small labelled SST-2 development split, so changes of one or two examples produce visible accuracy differences. Third, the custom models reconstruct quantized weights for FP32 Keras execution. The storage results therefore describe parameter representations and do not demonstrate latency, energy, throughput, or runtime-memory improvements.

VI. Conclusion

This study analyzed uniform per-channel PTQ and non-uniform codebook weight quantization in two transformer architectures, using ResNet-18 as a CNN reference. Across all three models, W8 preserved accuracy while reducing estimated parameter storage by approximately 75%. The central architecture-level result appears at W4: DistilBERT and ViT-B/16 remained within 0.6 percentage points of their FP32 baselines at about $7.9\times$ compression, whereas ResNet-18 collapsed to near-chance accuracy under both methods.

The low-bit sweeps also show that the quantization method matters most below four bits. Codebook quantization substantially outperformed PTQ for DistilBERT at W2 and for ViT at W3, but neither method provided uniformly reliable performance at the lowest precisions. A plausible, untested explanation for the 57.55-percentage-point ViT W3 gap is that learned codebook centroids adapt to a non-uniform weight distribution, whereas a three-bit uniform PTQ grid provides too few fixed levels to represent the same weights adequately. The isolated sensitivity analysis explains part of the CNN failure: ResNet’s first convolution was exceptionally vulnerable at W4, while the sampled transformer tensors and modules produced much smaller isolated losses. Cross-bit rank correlations were generally weak, so sensitivity observed at W8 should not be treated as a dependable proxy for W4 behavior.

Overall, the evaluated transformer models are more tolerant of four-bit weight quantization than the CNN reference under the stated selection and evaluation policies. This makes low-bit transformer storage promising, but the conclusion is limited to model accuracy and parameter representation. Future deployment work should execute genuinely packed low-bit kernels and measure latency, runtime memory, throughput, and energy on target hardware before claiming computational-efficiency gains.

References

- [1] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.
- [2] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” arXiv:1806.08342, 2018.
- [3] M. Nagel et al., “A white paper on neural network quantization,” arXiv:2106.08295, 2021.
- [4] A. Vaswani et al., “Attention is all you need,” in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
- [5] R. M. Gray, “Vector quantization,” IEEE ASSP Magazine, vol. 1, no. 2, pp. 4–29, 1984.
- [6] D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in Proc. 18th Annual ACM-SIAM Symp. Discrete Algorithms (SODA), 2007, pp. 1027–1035.
- [7] S. P. Lloyd, “Least squares quantization in PCM,” IEEE Trans. Information Theory, vol. 28, no. 2, pp. 129–137, 1982.
- [8] H. Jégou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 33, no. 1, pp. 117–128, 2011.

- [9] Y. Gong, L. Liu, M. Yang, and L. Bourdev, “Compressing deep convolutional networks using vector quantization,” arXiv:1412.6115, 2014.
- [10] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in Proc. International Conf. Learning Representations (ICLR), 2016.
- [11] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan, “Towards the limit of network quantization,” in Proc. ICLR Workshop on Efficient Methods for Deep Neural Networks, 2017.
- [12] J. Shao, H. Zhang, and J. Wu, “Diffusion product quantization,” arXiv:2411.12306, 2024.
- [13] J. Deng, S. Li, Z. Wang, H. Gu, K. Xu, and K. Huang, “VQ4ALL: Efficient neural network representation via a universal codebook,” arXiv:2412.06875, 2024.
- [14] H. A. Abushahla, D. Varam, A. J. N. Panopio, and M. I. AlHajri, “Quantized neural networks for microcontrollers: A comprehensive review of methods, platforms, and applications,” arXiv:2508.15008, 2025.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- [16] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Toronto, Canada, Tech. Rep., 2009.
- [17] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” arXiv:1910.01108, 2019.
- [18] R. Socher et al., “Recursive deep models for semantic compositionality over a sentiment treebank,” in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2013, pp. 1631–1642.
- [19] A. Wang et al., “GLUE: A multi-task benchmark and analysis platform for natural language understanding,” in Proc. International Conf. Learning Representations (ICLR), 2019.
- [20] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in Proc. International Conf. Learning Representations (ICLR), 2021.
- [21] T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.
- [22] Y. Benjamini and Y. Hochberg, “Controlling the false discovery rate: A practical and powerful approach to multiple testing,” *J. Royal Statistical Society B*, vol. 57, no. 1, pp. 289–300, 1995.